NEXT ENGINEER · _SHEERAT_

Data Structure

BCS301

One-day revision notes: definitions, solved examples, code and important questions

Program: B.Tech | **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS | **Semester:** 3 | **Credits:** 4

L-T-P: 3-1-0 | **Scheme:** AKTU B.Tech 2nd year, 2023-24 onwards

UNIT 1 Arrays, Complexity and Linked Lists

UNIT 2 Stacks, Recursion and Queues

UNIT 3 Searching, Hashing and Sorting

UNIT 4 Trees

UNIT 5 Graphs

Original revision notes written for the AKTU syllabus. Every unit has the same parts:

1. **2-mark definitions**: short answers you can write word for word in Section A.
2. **Core concepts**: what you need to understand, with formulas.
3. **Solved examples**: the numericals and dry runs that keep appearing.
4. **Code to remember**: short C functions examiners ask you to write.
5. **Important questions**: the long questions to prepare, with what your answer must contain.

How to use these notes the day before the paper

- **Morning (3 to 4 hours):** read Units 1 and 2, and redo every solved example on paper without looking.
- **Afternoon (3 to 4 hours):** Units 3 and 4. Trees carry the most marks: do the AVL, B-tree and traversal examples twice.
- **Evening (2 hours):** Unit 5, then the "Last-hour checklist" at the end.
- **In the exam:** draw a diagram for every tree, graph, stack or queue answer. Write the time complexity at the end of every algorithm. Give a dry run table wherever you can. These are where the marks are.

AKTU paper pattern (theory, 70 marks, 3 hours): Section A has 7 short questions of 2 marks, all compulsory (14). Section B has 5 questions of 7 marks, attempt any 3 (21). Section C has 5 questions (one per unit), each with two parts, attempt one part of each, 7 marks each (35). Every unit gets its own Section C question, so do not skip any unit.

UNIT 1 Arrays, Complexity and Linked Lists

✍️ 2-mark definitions

- **Data structure:** a way of storing and organising data in memory so that the required operations can be done efficiently.
- **Linear vs non-linear:** in a linear structure elements form a sequence (array, linked list, stack, queue); in a non-linear structure an element can connect to many others (tree, graph).
- **Algorithm:** a finite sequence of well-defined steps that takes input and produces output. Properties: input, output, definiteness, finiteness, effectiveness.
- **Time complexity:** the amount of time an algorithm takes, expressed as a function of input size n. **Space complexity:** the extra memory it needs as a function of n.
- **Abstract Data Type (ADT):** a type defined by its operations and their behaviour, without saying how it is implemented. Example: Stack ADT = push, pop, peek, isEmpty.
- **Time–space trade-off:** reducing running time by using more memory, or saving memory at the cost of more time. Example: a lookup table makes searching faster but takes space.
- **Sparse matrix:** a matrix in which most elements are zero, so only the non-zero elements are stored.
- **Linked list:** a linear collection of nodes where each node holds data and a pointer to the next node; elements need not be contiguous in memory.

💡 Core concepts

Asymptotic notations

Notation	Meaning	Condition
Big O, O(g(n))	Upper bound (worst case)	$f(n) \le c \cdot g(n)$ for all $n \ge n_o$
Big Omega, Ω(g(n))	Lower bound (best case)	$f(n) \ge c \cdot g(n)$ for all $n \ge n_o$
Big Theta, Θ(g(n))	Tight bound	$c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_o$

Growth order: **$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$**

How to find complexity from code:

- One loop running n times → O(n).
- Two nested loops, each n → $O(n^2)$.
- Loop where i doubles (`i = i * 2`) or halves → O(log n).
- Outer loop n, inner loop log n → O(n log n).

Example to show that $3n^2 + 5n + 2 = O(n^2)$: for $n \ge 1$, $3n^2 + 5n + 2 \le 3n^2 + 5n^2 + 2n^2 = 10n^2$. So $c = 10$, $n_o = 1$.

Array address formulas (B = base address, W = bytes per element, L = lower bound, U = upper bound)

- Number of elements in a dimension: U − L + 1.

- 1-D: `Loc(A[i]) = B + W(i - L)`
- 2-D **row-major**, with N columns: $Loc(A[i][j]) = B + W[N(i - L_1) + (j - L_2)]$
- 2-D **column-major**, with M rows: $Loc(A[i][j]) = B + W[M(j - L_2) + (i - L_1)]$
- 3-D row-major, sizes M × N × P: $Loc(A[i][j][k]) = B + W[(i - L_1)NP + (j - L_2)P + (k - L_3)]$
- C stores arrays in row-major order.

Linked list types

Type	Pointers per node	Advantage	Drawback
Singly	next	Simple, less memory	One direction only
Doubly	prev, next	Both directions; delete a known node in O(1)	Extra pointer per node
Circular	last → first	Any node can be the start; good for round-robin	Must be careful to stop traversal

Array vs linked list (frequent 2-mark/5-mark question)

Array	Linked list
Fixed size	Grows and shrinks at run time
Contiguous memory	Nodes anywhere in memory
Random access O(1)	Sequential access O(n)
Insert/delete in middle costs O(n) shifting	Insert/delete O(1) once position is known
No pointer overhead	Extra memory for pointers

🧮 Solved examples

Example 1 (address calculation). An array `A[1..10][1..15]` has base address 100 and each element takes 4 bytes. Find the address of `A[4][7]` in row-major and column-major order.

- Rows M = 10, columns N = 15, $L_1 = L_2 = 1$.
- Row-major: 100 + 4[15(4 − 1) + (7 − 1)] = 100 + 4[45 + 6] = 100 + 204 = **304**.
- Column-major: 100 + 4[10(7 − 1) + (4 − 1)] = 100 + 4[60 + 3] = 100 + 252 = **352**.

Example 2 (sparse matrix, triplet form).

```
Matrix (4 × 5):          Triplet form:
0 0 3 0 4                Row Col Value
0 0 5 7 0                 4   5    6     ← header: rows, cols, non-zeros
0 0 0 0 0                 0   2    3
0 2 6 0 0                 0   4    4
                          1   2    5
                          1   3    7
                          3   1    2
                          3   2    6
```

Storing 20 values is replaced by 7 rows of 3, and the saving grows with matrix size.

Example 3 (polynomial addition). $P = 5x^3 + 4x^2 + 2$ and $Q = 3x^3 + x + 7$.

- Compare exponents from the highest: x^3 terms match → $5 + 3 = 8x^3$.
- $4x^2$ has no partner → copy $4x^2$. x^1 only in Q → copy x. Constants → $2 + 7 = 9$.
- Result: **$8x^3 + 4x^2 + x + 9$**.

⌨️ Code to remember

```
struct node { int data; struct node *next; };

/* insert at beginning */
struct node* insertBegin(struct node *head, int x) {
    struct node *t = malloc(sizeof(struct node));
    t->data = x;
    t->next = head;
    return t;                          /* new head */
}

/* delete first node holding key */
struct node* deleteKey(struct node *head, int key) {
    struct node *p = head, *prev = NULL;
    while (p != NULL && p->data != key) { prev = p; p = p->next; }
    if (p == NULL) return head;        /* not found */
    if (prev == NULL) head = p->next; /* deleting the head */
    else prev->next = p->next;
    free(p);
    return head;
}

/* reverse a singly linked list: very common question */
struct node* reverse(struct node *head) {
    struct node *prev = NULL, *cur = head, *nxt;
    while (cur != NULL) {
        nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}
```

Polynomial node: `struct poly { int coef; int exp; struct poly *next; };`

⭐ Important questions

1. Define asymptotic notations with graphs. Show that a given polynomial is $O(n^2)$. *Include all three definitions, a sketch for each, and a worked proof with c and n_o.*
2. Derive the address formula for a 2-D array in row-major and column-major order, and solve a numerical. *Derivation plus a full numerical like Example 1.*

3. Write algorithms to insert and delete a node (beginning, end, given position) in a singly / doubly linked list. *Draw before-and-after pointer diagrams.*
4. Represent a polynomial using a linked list and write an algorithm to add two polynomials. *Node structure, algorithm, and one worked example.*
5. What is a sparse matrix? Give its array and linked representation. *Use a small matrix like Example 2.*

NEXT ENGINEER
sheerat

UNIT 2 Stacks, Recursion and Queues

✍️ 2-mark definitions

- **Stack:** a linear data structure where insertion and deletion happen at one end, the top. It follows **LIFO** (last in, first out).
- **Overflow / underflow:** pushing onto a full stack is overflow; popping from an empty stack is underflow.
- **Queue:** a linear data structure where insertion happens at the rear and deletion at the front. It follows **FIFO** (first in, first out).
- **Circular queue:** a queue in which the last position wraps around to the first, so freed space at the front is reused.
- **Deque:** a double-ended queue where insertion and deletion can happen at both ends.
- **Priority queue:** a queue where each element has a priority, and the element with the highest priority is removed first.
- **Recursion:** a function calling itself, with a base case that stops the calls.
- **Tail recursion:** recursion where the recursive call is the last statement, so nothing is left to do after it returns.

💡 Core concepts

Applications of stack: expression conversion and evaluation, function calls and recursion, undo in editors, browser back button, balanced-parentheses checking, DFS.

Applications of queue: CPU and printer scheduling, BFS, buffering (keyboard, I/O), call-centre waiting lines.

Operator precedence (highest first): `^` (right to left) → `* /` (left to right) → `+ -` (left to right).

Infix to postfix rules

1. Operand → write to output.
2. `(` → push.
3. `)` → pop to output until `(`; discard both brackets.
4. Operator → while the stack top has higher precedence, or equal precedence and the operator is left-associative, pop it to output. Then push the new operator.
5. At the end, pop everything left to output.

Infix to prefix: reverse the infix expression and swap `(` with `)`, convert to postfix, then reverse the result.

Circular queue conditions (size MAX, front = rear = −1 initially)

- Empty: `front == -1`
- Full: `(rear + 1) % MAX == front`
- Next position: `rear = (rear + 1) % MAX`

Iteration vs recursion

Recursion	Iteration
Function calls itself	Loop repeats statements
Uses the system stack, O(depth) memory	Constant extra memory
Shorter code for trees, Hanoi, divide and conquer	Usually faster, no call overhead
Risk of stack overflow	No such risk

🧮 Solved examples

Example 1 (infix to postfix). Convert `A + B * (C - D) / E`.

Symbol	Stack	Output
A		A
+	+	A
B	+	A B
*	+ *	A B
(	+ * (	A B
C	+ * (	A B C
−	+ * (−	A B C
D	+ * (−	A B C D
)	+ *	A B C D −
/	+ /	A B C D − *
E	+ /	A B C D − * E
end		**A B C D − * E / +**

At `/`, the `*` on the stack has equal precedence and is left-associative, so it is popped first.

Example 2 (evaluate postfix). Evaluate `5 6 2 + * 12 4 / -`.

Symbol	Action	Stack
5, 6, 2	push	5 6 2
+	6 + 2 = 8	5 8
*	5 × 8 = 40	40
12, 4	push	40 12 4
/	12 / 4 = 3	40 3
−	40 − 3 = 37	**37**

When you pop two values, the first one popped is the **right** operand.

Example 3 (Tower of Hanoi, n = 3, pegs A → C using B). Moves: A→C, A→B, C→B, A→C, B→A, B→C, A→C. Total = $2^3 - 1 = \mathbf{7}$. Recurrence: $T(n) = 2T(n-1) + 1$, so $T(n) = 2^n - 1$.

⌨️ Code to remember

```
#define MAX 50
int stack[MAX], top = -1;

void push(int x) {
    if (top == MAX - 1) { printf("Overflow"); return; }
    stack[++top] = x;
}
int pop() {
    if (top == -1) { printf("Underflow"); return -1; }
    return stack[top--];
}

/* circular queue */
int q[MAX], front = -1, rear = -1;
void enqueue(int x) {
    if ((rear + 1) % MAX == front) { printf("Queue full"); return; }
    if (front == -1) front = 0;
    rear = (rear + 1) % MAX;
    q[rear] = x;
}
int dequeue() {
    if (front == -1) { printf("Queue empty"); return -1; }
    int x = q[front];
    if (front == rear) front = rear = -1;   /* last element removed */
    else front = (front + 1) % MAX;
    return x;
}

/* Tower of Hanoi */
void hanoi(int n, char from, char to, char via) {
    if (n == 1) { printf("%c -> %c\n", from, to); return; }
    hanoi(n - 1, from, via, to);
    printf("%c -> %c\n", from, to);
    hanoi(n - 1, via, to, from);
}

/* recursive binary search */
int bsearch(int a[], int lo, int hi, int key) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == key) return mid;
    if (key < a[mid]) return bsearch(a, lo, mid - 1, key);
    return bsearch(a, mid + 1, hi, key);
}
```

⭐ Important questions

1. Convert an infix expression to postfix (and prefix) using a stack, showing the stack at each step. *Always use the three-column table above.*
2. Evaluate a postfix expression using a stack. *Table with each push/pop.*
3. What is a circular queue? Why is it better than a linear queue? Write insert and delete algorithms. *Explain the wasted-space problem with a diagram, then the code above.*
4. Explain recursion, tail recursion and removal of recursion. Write the Tower of Hanoi program and find the number of moves. *Include the recurrence and the $2^n - 1$ result.*
5. Implement a stack and a queue using linked lists. *Push = insert at head; enqueue at rear, dequeue at front.*
6. Write short notes on deque and priority queue. *Types of deque (input-restricted, output-restricted), and heap-based priority queue.*

NEXT ENGINEER _sheerat_

UNIT 3 Searching, Hashing and Sorting

✍️ 2-mark definitions

- **Searching:** finding whether an element is present in a collection and, if so, its position.
- **Hashing:** a technique that maps a key directly to a table position using a hash function, giving average O(1) search.
- **Hash function:** a function h(k) that converts a key into a table index.
- **Collision:** when two different keys get the same hash value.
- **Load factor (α):** number of stored keys ÷ table size.
- **Stable sort:** a sort in which equal keys keep their original relative order.
- **In-place sort:** a sort that needs only O(1) extra memory (apart from the input).
- **Internal vs external sorting:** internal sorting works on data that fits in main memory; external sorting handles data on disk.

💡 Core concepts

Searching methods

Method	Sorted data needed?	Best	Worst
Linear	No	O(1)	O(n)
Index sequential	Yes		about O(√n)
Binary	Yes	O(1)	O(log n)

Hash functions: division `h(k) = k mod m` (m prime is best), mid-square (square the key, take middle digits), folding (split the key into parts and add them).

Collision resolution

- **Separate chaining:** each slot holds a linked list of keys that hash there.
- **Linear probing:** `h(k, i) = (h(k) + i) mod m`. Simple, but causes primary clustering.
- **Quadratic probing:** `h(k, i) = (h(k) + i²) mod m`. Less clustering.
- **Double hashing:** `h(k, i) = (h₁(k) + i·h₂(k)) mod m`. Best spread.

Sorting summary (learn this table)

Algorithm	Best	Average	Worst	Space	Stable
Bubble	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion	O(n)	O(n²)	O(n²)	O(1)	Yes
Quick	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes

Algorithm	Best	Average	Worst	Space	Stable
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Radix	O(d(n + k))	O(d(n + k))	O(d(n + k))	O(n + k)	Yes

- Quick sort's worst case happens when the pivot is always the smallest or largest element, for example sorted input with the first or last element as pivot.
- Merge sort recurrence: T(n) = 2T(n/2) + n → O(n log n).

🧮 Solved examples

Example 1 (linear probing). Insert 23, 43, 13, 27 into a table of size 10 with h(k) = k mod 10.

- 23 → 3.
- 43 → 3 is taken → 4.
- 13 → 3 taken, 4 taken → 5.
- 27 → 7.

Index	0	1	2	3	4	5	6	7	8	9
Key				23	43	13		27		

Example 2 (quick sort partition, last element as pivot). Array [7, 2, 9, 4, 3], pivot = 3, i = −1.

- j = 0: 7 > 3, skip. j = 1: 2 ≤ 3 → i = 0, swap a[0], a[1] → [2, 7, 9, 4, 3].
- j = 2: 9 > 3. j = 3: 4 > 3.
- Place pivot: swap a[i+1] = a[1] with a[4] → **[2, 3, 9, 4, 7]**. The pivot 3 is now in its final position; sort [2] and [9, 4, 7] the same way.

Example 3 (merge sort). [38, 27, 43, 3]



Example 4 (insertion sort passes). [5, 2, 4, 6, 1]

- Pass 1: [2, 5, 4, 6, 1]
- Pass 2: [2, 4, 5, 6, 1]
- Pass 3: [2, 4, 5, 6, 1] (6 already in place)
- Pass 4: [1, 2, 4, 5, 6]

Example 5 (radix sort). [170, 45, 75, 90, 802, 24, 2, 66]

- By units digit: 170, 90, 802, 2, 24, 45, 75, 66

- By tens digit: 802, 2, 24, 45, 66, 170, 75, 90
- By hundreds digit: 2, 24, 45, 66, 75, 90, 170, 802

⌨️ Code to remember

```
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
}

int partition(int a[], int lo, int hi) {
    int pivot = a[hi], i = lo - 1, t;
    for (int j = lo; j < hi; j++)
        if (a[j] <= pivot) { i++; t = a[i]; a[i] = a[j]; a[j] = t; }
    t = a[i + 1]; a[i + 1] = a[hi]; a[hi] = t;
    return i + 1;
}
void quickSort(int a[], int lo, int hi) {
    if (lo < hi) {
        int p = partition(a, lo, hi);
        quickSort(a, lo, p - 1);
        quickSort(a, p + 1, hi);
    }
}

void merge(int a[], int lo, int mid, int hi) {
    int tmp[hi - lo + 1], i = lo, j = mid + 1, k = 0;
    while (i <= mid && j <= hi) tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= mid) tmp[k++] = a[i++];
    while (j <= hi)  tmp[k++] = a[j++];
    for (k = 0; k < hi - lo + 1; k++) a[lo + k] = tmp[k];
}
void mergeSort(int a[], int lo, int hi) {
    if (lo < hi) {
        int mid = (lo + hi) / 2;
        mergeSort(a, lo, mid);
        mergeSort(a, mid + 1, hi);
        merge(a, lo, mid, hi);
    }
}
```

⭐ Important questions

1. What is hashing? Explain hash functions and collision resolution techniques, and insert given keys using linear/quadratic probing. *Full table like Example 1.*
2. Write the quick sort algorithm, sort a given array, and analyse its best, average and worst cases. *Partition dry run + recurrences T(n) = 2T(n/2) + n and T(n) = T(n − 1) + n.*

3. Write the merge sort algorithm and show it on a given array. Why is it stable? *Tree diagram like Example 3.*
4. Explain heap sort with an example. *Build max-heap, then swap root with last and heapify, showing the array each time.*
5. Compare all sorting techniques. *Reproduce the summary table.*
6. Binary search vs linear search, with algorithm and complexity.

NEXT ENGINEER
sheerat

UNIT 4 Trees (highest-weightage unit)

✍️ 2-mark definitions

- **Tree:** a non-linear hierarchical structure with a root node, where every other node has exactly one parent.
- **Degree of a node:** number of children. **Leaf:** node with degree 0.
- **Height of a tree:** the number of edges on the longest path from the root to a leaf.
- **Binary tree:** a tree where every node has at most two children.
- **Strictly binary tree:** every node has either 0 or 2 children.
- **Complete binary tree:** all levels are full except possibly the last, which is filled from left to right.
- **Extended binary tree (2-tree):** a binary tree where every empty subtree is replaced by a special external node, so every internal node has two children.
- **Binary search tree:** a binary tree where every node's left subtree has smaller keys and right subtree has larger keys.
- **Threaded binary tree:** a binary tree where NULL child pointers are replaced by pointers (threads) to the inorder predecessor or successor.
- **AVL tree:** a height-balanced BST where the balance factor of every node is −1, 0 or +1.
- **B-tree:** a balanced m-way search tree in which all leaves are at the same level, used for disk-based indexing.
- **Heap:** a complete binary tree where each parent is ≥ its children (max-heap) or ≤ its children (min-heap).

💡 Core concepts

Binary tree facts

- Maximum nodes at level l (root at level 0): 2^l.
- Maximum nodes in a binary tree of height h: $2^{h+1} - 1$.
- Minimum height of a tree with n nodes: $\lceil \log_2(n + 1) \rceil - 1$.
- Leaves = (nodes with two children) + 1.

Array representation (root at index 1): left child of i at 2i, right child at 2i + 1, parent at $\lfloor i/2 \rfloor$.

Traversals

- Preorder: **Root – Left – Right**
- Inorder: **Left – Root – Right** (gives sorted order for a BST)
- Postorder: **Left – Right – Root**
- A tree is uniquely built from **inorder + preorder** or **inorder + postorder**. Preorder + postorder alone is not enough (except for strictly binary trees).

BST deletion cases

1. Leaf: just remove it.
2. One child: connect the parent directly to the child.

3. Two children: replace the key with its **inorder successor** (smallest key in the right subtree), then delete that successor.

AVL rotations (balance factor = height(left) − height(right))

Case	Where the new node went	Fix
LL	Left subtree of left child	Single right rotation
RR	Right subtree of right child	Single left rotation
LR	Right subtree of left child	Left rotation on child, then right rotation
RL	Left subtree of right child	Right rotation on child, then left rotation

B-tree of order m rules: at most m children and m − 1 keys per node; every non-root internal node has at least ⌈m/2⌉ children; root has at least 2 children (unless it is a leaf); all leaves on the same level. Insert into a leaf; on overflow split the node and send the middle key up.

🧮 Solved examples

Example 1 (build tree from traversals). Inorder: D B E A F C G. Preorder: A B D E C F G.

- Preorder starts with A → root A. In inorder, left of A is {D B E}, right is {F C G}.
- Left part: next in preorder is B → root B, left D, right E.
- Right part: next in preorder is C → root C, left F, right G.



Postorder: **D E B F G C A**.

Example 2 (AVL insertion). Insert 10, 20, 30, 25, 28.

- 10, 20, 30: node 10 gets balance −2 (RR case) → left rotation → 20 becomes root.



- Insert 25 (left of 30), then 28 (right of 25). Node 30 now has balance +2 and the new node is in the right subtree of its left child → **LR case**: rotate left at 25, then right at 30.



Example 3 (B-tree of order 3, at most 2 keys per node). Insert 10, 20, 30, 40, 50.

- 10, 20 → [10 20]. 30 → [10 20 30] overflows → split, 20 goes up: root [20], children [10] [30].
- 40 → [30 40]. 50 → [30 40 50] overflows → split, 40 goes up into the root.

```
       [20 40]
      /   |   \
   [10] [30] [50]
```

Example 4 (Huffman coding). Frequencies: a 5, b 9, c 12, d 13, e 16, f 45.

- Join the two smallest each time: (a5 + b9) = 14; (c12 + d13) = 25; (14 + e16) = 30; (25 + 30) = 55; (f45 + 55) = 100.
- Label left edges 0 and right edges 1. Codes: **f = 0, c = 100, d = 101, a = 1100, b = 1101, e = 111**.
- Total bits = 45×1 + 12×3 + 13×3 + 5×4 + 9×4 + 16×3 = **224**, compared with 300 using a fixed 3-bit code.

Example 5 (build a max-heap by inserting 10, 20, 15, 30, 40).

- 10 → [10]. 20 → sift up → [20, 10]. 15 → [20, 10, 15].
- 30 → at index 4, parent 10 → swap → parent 20 → swap → [30, 20, 15, 10].
- 40 → at index 5, parent 20 → swap → parent 30 → swap → **[40, 30, 15, 10, 20]**.

⌨️ Code to remember

```
struct tnode { int key; struct tnode *left, *right; };

void inorder(struct tnode *r)   { if (r) { inorder(r->left); printf("%d ", r->key); inorder(r->right); } }
void preorder(struct tnode *r)  { if (r) { printf("%d ", r->key); preorder(r->left); preorder(r->right); } }
void postorder(struct tnode *r) { if (r) { postorder(r->left); postorder(r->right); printf("%d ", r->key); } }

struct tnode* insertBST(struct tnode *r, int x) {
    if (r == NULL) {
        r = malloc(sizeof(struct tnode));
        r->key = x; r->left = r->right = NULL;
    } else if (x < r->key) r->left = insertBST(r->left, x);
    else if (x > r->key)   r->right = insertBST(r->right, x);
    return r;
}

int height(struct tnode *r) {
    if (r == NULL) return -1;
    int l = height(r->left), h = height(r->right);
    return 1 + (l > h ? l : h);
}
```

⭐ Important questions

1. Construct a binary tree from given inorder and preorder (or postorder) traversals and write the third traversal. *Step-by-step splitting like Example 1.*
2. Insert a sequence of keys into an AVL tree, showing the rotation at every imbalance. *Name the case (LL/RR/LR/RL) each time and redraw the tree.*
3. Insert/delete keys in a B-tree of a given order. *Show every split; state the B-tree rules first.*
4. Write algorithms for insertion, deletion and searching in a BST, with all three deletion cases. *Diagram for each case.*
5. What is a threaded binary tree? Why is it needed? Show inorder traversal without a stack. *Diagram with dotted threads and the flag fields.*
6. Explain Huffman coding and build the tree for given frequencies. *Like Example 4, with the final code table and total bits.*

NEXT ENGINEER _sheerat_

UNIT 5 Graphs

✍️ 2-mark definitions

- **Graph:** a set of vertices V and a set of edges E connecting pairs of vertices, written G = (V, E).
- **Directed graph:** edges have a direction (u → v). **Undirected graph:** edges have no direction.
- **Weighted graph:** each edge has a cost or weight.
- **Degree:** number of edges at a vertex. In a directed graph, **in-degree** counts incoming and **out-degree** outgoing edges.
- **Path:** a sequence of vertices joined by edges. **Cycle:** a path that starts and ends at the same vertex.
- **Connected graph:** there is a path between every pair of vertices.
- **Complete graph:** every pair of vertices has an edge; it has n(n − 1)/2 edges (undirected).
- **Spanning tree:** a subgraph that includes all vertices, is connected and has no cycle; it has exactly V − 1 edges.
- **Minimum spanning tree (MST):** a spanning tree with the smallest total edge weight.
- **Transitive closure:** a matrix that says, for every pair (i, j), whether a path exists from i to j.

💡 Core concepts

Representations

	Adjacency matrix	Adjacency list
Space	$O(V^2)$	O(V + E)
Check edge (u, v)	O(1)	O(degree of u)
Best for	Dense graphs	Sparse graphs

BFS vs DFS

BFS	DFS
Uses a queue	Uses a stack or recursion
Visits level by level	Goes deep first, then backtracks
Gives shortest path in unweighted graphs	Used for cycle detection, topological sort
O(V + E)	O(V + E)

Prim vs Kruskal

Prim	Kruskal
Grows one tree from a start vertex	Adds the cheapest edges anywhere, merging forests
Picks the cheapest edge leaving the tree	Sorts all edges, skips ones that form a cycle

Prim	Kruskal
Better for dense graphs	Better for sparse graphs
O(E log V) with a heap	O(E log E)

Dijkstra works only with non-negative edge weights. Relaxation: if `d[u] + w(u, v) < d[v]`, set `d[v] = d[u] + w(u, v)`.

Warshall's algorithm (transitive closure): for k = 1..n, for i, for j: `R[i][j] = R[i][j] OR (R[i][k] AND R[k][j])`. $O(n^3)$.

🧮 Solved examples

Use this graph for all the examples below (undirected, weighted):



Edges: A–B 4, A–C 1, B–C 2, B–D 5, C–D 8, D–E 3.

Example 1 (BFS and DFS from A, neighbours taken in alphabetical order).

- BFS: queue A → visit A, enqueue B, C → visit B, enqueue D → visit C → visit D, enqueue E → visit E. Order: **A B C D E**.
- DFS: A → B (first neighbour of A) → C (first unvisited neighbour of B) → D (C's next unvisited) → E. Backtrack. Order: **A B C D E**. (Neighbours of B in order are A, C, D; since C comes before D, DFS goes to C.)

Example 2 (Kruskal). Sorted edges: A–C 1, B–C 2, D–E 3, A–B 4, B–D 5, C–D 8.

- Take A–C (1). Take B–C (2). Take D–E (3). A–B (4) makes cycle A–B–C → skip. Take B–D (5). Now 4 edges for 5 vertices → done.
- MST edges: A–C, B–C, D–E, B–D. **Total = 11.**

Example 3 (Prim from A).

- Tree {A}: cheapest leaving edge A–C 1 → add C.
- Tree {A, C}: options A–B 4, C–B 2, C–D 8 → add C–B 2.
- Tree {A, B, C}: options B–D 5, C–D 8 → add B–D 5.
- Tree {A, B, C, D}: option D–E 3 → add.
- **Total = 1 + 2 + 5 + 3 = 11**, the same as Kruskal.

Example 4 (Dijkstra from A).

Step	Visited	A	B	C	D	E
Start		0	∞	∞	∞	∞

Step	Visited	A	B	C	D	E
Visit A	A	0	4	1	∞	∞
Visit C	A, C	0	3	1	9	∞
Visit B	A, C, B	0	3	1	8	∞
Visit D	A, C, B, D	0	3	1	8	11
Visit E	all	0	3	1	8	11

Shortest distances: **B = 3 (A→C→B), C = 1, D = 8 (A→C→B→D), E = 11**.

⌨️ Code to remember

```
int adj[MAX][MAX], visited[MAX], n;

void dfs(int v) {
    visited[v] = 1;
    printf("%d ", v);
    for (int w = 0; w < n; w++)
        if (adj[v][w] && !visited[w]) dfs(w);
}

void bfs(int s) {
    int q[MAX], f = 0, r = 0;
    visited[s] = 1; q[r++] = s;
    while (f < r) {
        int v = q[f++];
        printf("%d ", v);
        for (int w = 0; w < n; w++)
            if (adj[v][w] && !visited[w]) { visited[w] = 1; q[r++] = w; }
    }
}
```

⭐ Important questions

1. Explain graph representations (adjacency matrix, adjacency list) with a diagram, and compare them.
2. Write BFS and DFS algorithms and give the traversal order for a given graph. *Show the queue/stack contents step by step.*
3. Find the MST of a given graph using Prim's and Kruskal's algorithms. *Show each step and the final weight for both.*
4. Write Dijkstra's algorithm and apply it to a given graph. *Distance table like Example 4.*
5. Explain Warshall's algorithm for transitive closure with an example. *Show the matrix after each k.*

Last-hour checklist

Read only this in the last hour before the paper.

- ☐ Big O, Ω, Θ definitions and the growth order.
- ☐ Row-major and column-major address formulas.
- ☐ Linked list insert/delete/reverse code.
- ☐ Infix → postfix rules and precedence table.
- ☐ Circular queue full/empty conditions.
- ☐ Tower of Hanoi: $2^n - 1$ moves.
- ☐ Sorting comparison table (best/average/worst/stable).
- ☐ Linear probing table method.
- ☐ Tree from inorder + preorder.
- ☐ AVL four rotation cases.
- ☐ B-tree split rule.
- ☐ Huffman: join the two smallest each time.
- ☐ BFS = queue, DFS = stack.
- ☐ Prim vs Kruskal, Dijkstra table.

Books prescribed by AKTU

These are the books named in the official AKTU syllabus. Buy or borrow them through a bookshop or your college library. The WorldCat links show which libraries hold a copy.

#	Title	Author(s)	Publisher	Where to get it legally
1	Data Structures Using C and C++	Aaron M. Tenenbaum, Yedidyah Langsam, Moshe J. Augenstein	PHI Learning	WorldCat
2	Data Structures: A Pseudocode Approach with C (3rd ed.)	Richard F. Gilberg, Behrouz A. Forouzan	Cengage Learning	WorldCat
3	Fundamentals of Data Structures	Ellis Horowitz, Sartaj Sahni	Galgotia Publications	WorldCat
4	Data Structures (Schaum's Outline Series)	Seymour Lipschutz	Tata McGraw-Hill	WorldCat
5	Data Structures Using C	Reema Thareja	Oxford University Press (Higher Education)	WorldCat
6	Data Structure Using C	A. K. Sharma	Pearson Education India	WorldCat

#	Title	Author(s)	Publisher	Where to get it legally
7	Data Structure Using C and C++	Rajesh K. Shukla	Wiley Dreamtech	WorldCat
8	Data Structures and Algorithms in C++	Michael T. Goodrich, Roberto Tamassia, David M. Mount	Wiley India	WorldCat
9	C and Data Structures	P. S. Deshpande	Wiley Dreamtech	WorldCat
10	Data Structures and Program Design in C	Robert Kruse et al.	Pearson Education	WorldCat
11	Data Structures: Theory and Practice	A. T. Berztiss	Academic Press	WorldCat
12	An Introduction to Data Structures with Applications	Jean-Paul Tremblay, Paul G. Sorenson	McGraw-Hill	WorldCat
13	Data Structures and Algorithms in Java	Adam Drozdek	Cengage Learning	WorldCat

Official syllabus: AKTU B.Tech 2nd year CSE syllabus, 2023-24 scheme (PDF)

NEXT ENGINEER _sheerat_